

A Self-Organizing Graph Runtime for Coupled Meaning and Resource Flow

Executive summary

This report formalizes a **graph runtime** in which (i) all persisted knowledge and provenance live in an append-only **TruthGraph**, (ii) computation and control operate on a **losslessly compressed ViewGraph** that is dynamically expanded/compacted under demand, and (iii) system behavior emerges from **Presences** (agents with internal state only) exchanging **Daimoi** (probabilistic message packets) and participating in a **resource-fluid economy** with pressure-derived local prices and **need/mass gravity wells** defined over **graph distance**.

The key conceptual choices are:

- **Semantics-as-topology**: embeddings and “meaning” are reflected in where edges/nodes exist in the ViewGraph, enabling **auditable congregation** via shortest-path structure rather than hidden global indexing. Shortest-path distance is computed using standard algorithms and can be inspected path-by-path. ¹
- **Resources-as-fluid**: compute/IO resources are simulated as continuous flows through reservoirs with pressure and capacity, while prices are derived from local scarcity and congestion, consistent with classical formulations where “prices” act as decentralized signals for allocation and stability. ²
- **Messages-as-uncertainty packets**: a Daimoi carries a **distribution over possible components**; when absorbed, a Presence **samples** a component, biased by its purpose embedding (“lens”) and its current needs—turning interpretation into a contextual stochastic policy.
- **Learning-from-outcomes**: the lens strength and sampling temperature are learned online with **policy-gradient / REINFORCE-style updates** and eligibility-weighted credit assignment. ³
- **Compression-as-governance**: Sentinels decide when to compact/expand; Compactors execute the reversible transform. The ViewGraph is a demand-driven projection of TruthGraph, analogous to projections/materialized views over an immutable event log. ⁴
- **Ephemeral nodes are “high-resolution views,” not loss**: ephemeral ViewGraph nodes (streams, events, temporary bridges) are implemented as reversible windows onto TruthGraph event chains and are consolidated when cold.

This document provides (1) precise definitions and notation; (2) core algorithms (Daimoi emission/mixing/drift, biased sampling, traces and credit assignment, online learning); (3) hybrid fluid/particle simulation with routing and bounded-radius gravity computation; (4) an evaluation plan (metrics, workloads, ablations, diagnostics); (5) implementation schemas and scalability analysis; (6) recommended priors and safety caps; and (7) a prioritized reading list anchored in primary sources on shortest paths, congestion control, network utility pricing, policy gradients, and graph summarization/compression. ⁵

Formal abstractions and notation

Graph layers

Let $G^T = (V^T, E^T)$ be the **TruthGraph** and $G^V = (V^V, E^V)$ be the **ViewGraph**.

- **TruthGraph (persistence)**: append-only, lossless store of all content and events. Every node/edge in G^T is immutable after creation; updates are modeled as new events/edges.
- **ViewGraph (simulation)**: a losslessly compressed projection used for routing, flow, and control decisions. ViewGraph nodes may represent either atomic truth items or **bundles**.

Lossless projection. Maintain a surjective map $\Pi : V^T \rightarrow V^V$. Each ViewGraph bundle node $b \in V^V$ carries a **reconstruction ledger** sufficient to recover the induced subgraph on $\Pi^{-1}(b)$ and all boundary relations. This is aligned with the broad taxonomy of graph summarization where summaries may be lossless if they preserve the information required to reconstruct or answer exact queries, and where “summaries” can be treated as derived structures rather than destructive transformations. 6

Nexus nodes

A **Nexus** is a node $n \in V^V$ which may represent:

- a document/media artifact (e.g., MP3),
- a resource reservoir (CPU tokens),
- a gateway/connector (cross-basin edges),
- an event/stream window (ephemeral projection).

Each Nexus node can host **resource reservoirs** and **routing ports** but has no agency.

Resource types and reservoir dynamics

Let resource types be $k \in \mathcal{K}$ (CPU/GPU/NPU/RAM/DISK/NETUP/NETDOWN).

State variables per node/resource:

- $R[n, k] \geq 0$: resource amount stored
- $Cap[n, k] > 0$: capacity
- $P[n, k]$: pressure (monotone in fill fraction)

A canonical smooth pressure law:

$$P[n, k] = P_k^{\max} \cdot \phi\left(\frac{R[n, k]}{Cap[n, k]}\right), \quad \phi(x) = \min(1, x)^\beta, \quad \beta > 1.$$

This “convex near-full” behavior is used to create stronger gradients as reservoirs approach limits (design choice; β is a hyperparameter).

Edges $e = (u \rightarrow v) \in E^V$ carry per-resource flow $F[e, k]$ bounded by throughput $F_{\max}[e, k]$.

Edge congestion, affinity, and graph-distance cost

Let:

- $sat_e \in [0, 1]$: edge saturation (e.g., $sat_e = \max_k |F[e, k]| / F_{\max}[e, k]$)
- $a_e \in [0, 1]$: semantic affinity (how “meaningfully short” the corridor is)

Define a traversal cost used for graph distance:

$$cost(e) = w_L + w_C \cdot sat_e + w_S \cdot (1 - a_e), \quad w_L, w_C, w_S \geq 0.$$

Shortest paths are computed using standard shortest-path methods; auditable routing follows directly from the inspected path and its edge costs. ¹

Presences, masks, influence, need, and mass

A **Presence** $p \in \mathcal{P}$ is the only actor. It owns only internal state and can emit/absorb Daimoi.

Mask. A Presence occupies the graph probabilistically via a sparse distribution $M_p : V^V \rightarrow [0, 1]$ with $\sum_n M_p(n) = 1$. Masked expectation:

$$\langle g \rangle_{M_p} = \sum_{n \in V^V} M_p(n) g(n).$$

Influence. Define $infl(p, n) \in [0, 1]$ (often derived from mask weight plus explicit pinning/sponsorship). Influence weights are used to compute Presence “mass” and to select representative “source nodes” for distance calculations.

Need. For each k , let $u_{p,k} \in [0, 1]$ be a utilization EMA (or scarcity proxy). Convert to need:

$$need_{p,k} = priority_p \cdot \sigma(a_k(u_{p,k} - \theta_k)),$$

with logistic σ , threshold θ_k , steepness a_k . This makes need saturating and prevents unbounded “black holes” from small fluctuations.

Mass. Let node mass combine informational and resource significance:

$$m(n) = m_{info} \log(1 + \text{bytes}(n)) + \sum_k m_k \cdot \frac{R[n, k]}{Cap[n, k] + \epsilon}.$$

Presence mass:

$$Mass_p = m_0 + \sum_{n \in V^V} infl(p, n) m(n).$$

Daimoi packets

A Daimoi $d \in \mathcal{D}$ is a probabilistic packet over components c_i :

- components $c_1, \dots, c_M$
- categorical probabilities $p_1, \dots, p_M$ with $\sum_i p_i = 1$
- embeddings $\mathbf{e}_i$ per component
- size estimate $size(c_i)$
- resource signature $req(c_i, k) \geq 0$ indicating how component i relates to resource k
- packet energy $E(d)$ (size-cost proxy)
- amplitude/frequency (A, f) indicating consolidation of repeated events

Packet expected resource signature:

$$\rho_{d,k} = \sum_i p_i req(c_i, k).$$

Gravity potentials and maps

Use **graph distance** (not embedding distance) to define Presence potentials:

Let $dist(p, n)$ be the shortest path distance from a set of Presence sources $Sources(p) \subset V^V$ to node n , using $cost(e)$ above. Shortest-path computations are standard and auditable. ¹

Presence potential:

$$\Phi_p(n) = G_0 \cdot \frac{Mass_p}{dist(p, n)^2 + \epsilon}.$$

Aggregate per-resource gravity map:

$$G_k[n] = \sum_{p \in \mathcal{P}} need_{p,k} \Phi_p(n),$$

computed approximately via bounded-radius neighborhoods (algorithm below).

Pricing and CostCredits

Define normalized pressure $\hat{P}[n, k] = \text{clamp}(P[n, k]/P_k^{\max}, 0, 1)$.

Local unit price:

$$\pi_k(n) = \exp(\kappa_k \hat{P}[n, k]) \cdot (1 + \rho_k sat(n)),$$

where $sat(n)$ is local port congestion (e.g., average of incident sat_e).

Presence-local effective price:

$$\pi_{p,k}^{eff} = \langle \pi_k \rangle_{M_p} \cdot m_{p,k}, \quad m_{p,k} = \exp(u_{p,k}), \quad u_{p,k} \in [-u_{\max}, u_{\max}].$$

Given a resource spend vector Δ_k , define a dimensionless fractional spend using local availability $Avail_{p,k} = \langle Cap[\cdot, k] \rangle_{M_p}$:

$$\delta_k = \text{clamp} \left(\frac{\Delta_k}{\max(Avail_{p,k}, \epsilon)}, 0, \delta_{\max} \right).$$

Scalar **CostCredits**:

$$CostCredits(\Delta; p) = \sum_k \pi_{p,k}^{eff} \delta_k.$$

This resembles the role of prices/shadow prices in decentralized network resource allocation and congestion control, where local congestion translates to “cost signals” guiding distributed adaptation. ²

Ephemeral nodes as lossless projections of event chains

Ephemeral ViewGraph nodes represent **windows** onto TruthGraph event logs. TruthGraph event sourcing patterns treat all state changes as append-only events and reconstruct state via replay and projections. ⁷

- In G^T , events form chains (per stream) and optional causal links; log-based systems emphasize append-only ingestion and replayable consumers. ⁸
- In G^V , ephemeral nodes are high-resolution “materializations” that are compacted into bundles when cold; losslessness is ensured by retaining chain boundaries + cryptographic digests (e.g., Merkle-style) for verification. ⁹

Minimal μ -style contracts

```
(viewgraph.contract.v0
  (node Nexus
    (fields id kind embedding_ref truth_ptrs)
    (reservoir (R[k] Cap[k] P[k]))
    (ports (edges incident) (local_sat sat)))

  (edge Pipe
    (fields src dst affinity a_e health H_e)
    (per_resource (F[k] Fmax[k] conductance kappa[k]))
    (derived sat_e cost_e))

  (presence Contract
    (fields id kind purpose_embed L priority)
    (mask (sparse (node_id weight)))
    (influence infl(p,n))
    (need need[k] util_ema[k])
    (wallet budget[k]))
```

```
(learning (w_beta w_T u_pk baseline normalizer trace_buffer)))

(daimoi Packet
  (fields id origin mask_snapshot energy E amp A freq f ttl)
  (components (list (type embedding req[k] size payload_ref p_i))))
```

Daimoi lifecycle and learning algorithms

Daimoi creation and consolidation

Emission (Presence-only). A Presence emits a Daimoi when it produces an observation/action request/receipt. Components are typically single-mode at emission and become mixed by consolidation and network pressure.

A simple emission initializer using purpose embedding $\mathbf{L}_p$:

$$p_i^{emit} = \text{softmax}(\gamma \cdot \text{sim}(\mathbf{L}_p, \mathbf{e}_i) + b_i).$$

Energy budget. Use expected energy:

$$E(d) = \sum_i p_i \alpha \cdot \text{size}(c_i)^\gamma, \quad \gamma \geq 1.$$

Consolidation. When two packets d_1, d_2 merge (same corridor/time window), combine distributions energy-weightedly:

$$\mathbf{p} = \frac{E_1 \mathbf{p}_1 + E_2 \mathbf{p}_2}{E_1 + E_2}, \quad E = E_1 + E_2.$$

Amplitude A can scale sublinearly with repeat count (e.g., $\log(1 + n)$) to preserve “frequency” without explosive mass.

Daimoi drift and inter-graph discovery

Daimoi perform a biased stochastic walk on G^V (or hop-by-hop routing) guided by:

- gravity gradients $G_k[m] - G_k[n]$,
- traversal costs $\text{cost}(e)$,
- optionally, congestion penalties.

At node n , choose outgoing neighbor m with:

$$Pr(n \rightarrow m) \propto \exp \left(\eta \sum_k \rho_{d,k} (G_k[m] - G_k[n]) - v \cdot \text{cost}(n \rightarrow m) \right).$$

When “local not found” intent pressure rises, gates/bridges open as relief valves (governance policy) to allow Daimoi to navigate to other basins; these basins are explicit via connector Nexus nodes. This keeps movement auditable because every cross-basin step traverses concrete edges.

Absorption as biased sampling through a learned lens

When Presence p absorbs Daimoi d , it **samples** a component i^* from a biased distribution:

- lens similarity $s_i = \text{sim}(\mathbf{L}_p, \mathbf{e}_i)$
- need match $q_i = \sum_k \text{need}_{p,k} \text{req}(c_i, k)$

Let x be a feature vector (local congestion, prices, wallet, message entropy, etc.) and $\hat{x}$ its normalized form.

Learned parameters:

$$\beta(x) = \text{clip}(\text{softplus}(w_\beta^\top \hat{x}), 0, \beta_{\max})$$

$$T(x) = \text{clip}(T_{\min} + \text{softplus}(w_T^\top \hat{x}), T_{\min}, T_{\max})$$

Logits:

$$\ell_i = \log p_i + \beta(x) s_i + \zeta q_i - \lambda \text{cost}(c_i)$$

Biased sampling distribution:

$$\tilde{p}_i = \text{softmax}\left(\frac{\ell_i}{T(x)}\right).$$

Then sample $i^* \sim \text{Categorical}(\tilde{\mathbf{p}})$.

Implementation note (sampling): categorical sampling can be implemented with the Gumbel-max trick; extensions and relaxations (Gumbel-Softmax / Concrete) are well studied. 10

Absorption traces and eligibility credit assignment

Each absorption is a “decision” for learning. Store a trace τ :

- $\tau.time$
- $\tau.\text{logp} = \log \tilde{p}_{i^*}$
- $\tau.\hat{x}$
- $\tau.scope$ (mask snapshot / region identifiers)
- optional: a hash of the Daimoi id and realized component id

When an outcome event yields reward $r(t)$, distribute credit to traces using eligibility:

$$\text{elig}(\tau, t) = \exp\left(-\frac{t - \tau.time}{\tau_e}\right).$$

If causal linkage is uncertain, multiply by overlap between outcome scope and trace scope:

$$w_\tau \propto \text{elig}(\tau, t) \cdot \text{overlap}(\tau.\text{scope}, \text{event}.\text{scope}).$$

Eligibility traces are a standard mechanism for bridging delayed rewards and incremental updates; policy-gradient methods commonly combine baselines and eligibility for stability. ¹¹

Online learning updates for β , T , and price adapters

Use REINFORCE-style updates (stochastic policy gradient): ¹²

$$\theta \leftarrow \theta + \eta \cdot (r - \bar{r}) \cdot \nabla_\theta \log \tilde{p}_i^*,$$

with baseline $\bar{r}$ (EMA) and optional variance normalization.

Parameters θ include w_β, w_T . Additional adapters $u_{p,k}$ (price sensitivity multipliers) can be learned with the same signal but tightly bounded to preserve global price semantics.

Bandit and online learning references provide design guidance for nonstationary rewards, baselines, and exploration control. ¹³

Reproducible pseudocode

```
def absorb(p, d, now):
    x = features(p, d)                # pressures, wallet, sat, msg entropy, etc.
    xhat = ema_zscore(p.norm, x)

    beta = clip(softplus(dot(p.w_beta, xhat)), 0, BETA_MAX)
    temp = clip(T_MIN + softplus(dot(p.w_T, xhat)), T_MIN, T_MAX)

    logits = []
    for i, c in enumerate(d.components):
        s = cosine(p.L, c.embed)
        q = sum(p.need[k] * c.req[k] for k in K)
        logits.append((log(d.p[i]) + beta*s + ZETA*q - LAMBDA*c.size_cost) /
temp)

    i_star = categorical_sample_softmax(logits) # (e.g., Gumbel-max)
    p.traces.push(Trace(now, log_softmax(logits)[i_star], xhat, scope=p.scope))
    return i_star

def learn(p, reward, now):
    p.r_bar = ema(p.r_bar, reward)
    for tau in p.traces.recent():
        elig = exp(-(now - tau.time)/TAU_E)
        adv = (reward*elig - p.r_bar)
        grad = grad_logp(tau, p)      # compute from saved xhat + current params
```



```
p.params += LR * adv * grad
p.traces.gc(ttl=TRACE_TTL, max_len=TRACE_MAX)
```

Hybrid simulation and governance

Hybrid fluid/particle model

The system couples:

- **Fluid simulation** (high-volume): resources flow continuously on edges between node reservoirs, driven by pressure differences, capped by edge throughput.
- **Particle simulation** (low-volume): Daimoi represent discrete control/information packets and can consolidate.

This separation reduces computational load and improves auditability: the macrostate (pressures, prices, gravity maps) can be inspected independently from microstate (packet routes, sampling outcomes).

Fluid update and routing/valve scoring with gravity

A baseline edge flow:

$$F[e, k] = \text{clip}\left(\kappa[e, k] \cdot (P[u, k] - P[v, k]), [-F_{\max}[e, k], F_{\max}[e, k]]\right).$$

To make routing adaptive, set conductances $\kappa[e, k]$ via a softmax over outgoing edges from node u :

$$\begin{aligned} \text{score}(e, k) &= \alpha_P(P[u, k] - P[v, k]) + \alpha_G(G_k[v] - G_k[u]) + \alpha_A a_e - \alpha_S \text{sat}_e, \\ \kappa[e, k] &\propto \text{softmax}_{e \in \text{out}(u)}(\text{score}(e, k)). \end{aligned}$$

The gravity term biases flow “downhill” in the aggregated demand potential, while the congestion term discourages overloading, conceptually aligned with congestion control principles where local congestion signals modulate sending rates. ¹⁴

Bounded-radius gravity map computation

Directly computing $G_k[n]$ for all p, n is too expensive. Use bounded neighborhoods:

1. Choose $Sources(p)$ as top- S nodes by $\text{infl}(p, n)$.
2. Run multi-source shortest paths from $Sources(p)$ with edge costs $\text{cost}(e)$, stopping when distance exceeds R (hop radius or cost radius).
3. Accumulate $\Phi_p(n)$ inside the explored set.
4. Add to $G_k[n]$ only if $\text{need}_{p,k} > \text{need}_{\min}$.

Shortest-path procedures are standard and well characterized. ¹

```

def update_gravity(G, presences):
    for k in K: G.Gk[k].clear()

    for p in presences:
        needs = p.need
        if max(needs.values()) <= NEED_MIN:
            continue

        sources = top_sources(p, S_SOURCES)
        dist = bounded_multi_source_dijkstra(G, sources, R_COST)

        mass = p.mass
        for n, d in dist.items():
            phi = G_CONST * mass / (d*d + EPS)
            for k in K:
                if needs[k] > NEED_MIN:
                    G.Gk[k][n] += needs[k] * phi

```

Compaction/expansion via Sentinels and Compactors

Sentinel computes when to compact/expand based on localized metrics:

- demand intensity (queries, packet absorption rate),
- pressure variance,
- congestion levels,
- oscillation signals (thrash rate between regimes).

Compactor performs reversible transforms:

- compress: replace a subgraph with a bundle node plus a reconstruction ledger,
- expand: materialize the subgraph exactly from the ledger/truth pointers.

Graph coarsening methods and multilevel approaches are well established for producing smaller graphs while maintaining structure relevant to downstream operations. ¹⁵

The “lossless” requirement here is stronger than typical partitioning/summarization; thus, the reconstruction ledger must retain all information necessary to replay. This concept aligns with event-log/projection patterns where derived views are rebuilt from append-only histories. ¹⁶

Ephemeral connectors and completion signals

Ephemeral nodes serve as **temporary high-conductance channels** or **stream windows** in ViewGraph, while TruthGraph retains the underlying event chain.

- A “play track” request creates a temporary session/bridge Nexus in ViewGraph.

- The audio actor emits a completion event into TruthGraph and a completion Daimoi into the ephemeral bridge for immediate return.
- When the bridge expires, it is compacted into a bundle/receipt that references the TruthGraph chain segment (optionally with a Merkle-style digest for audit). ¹⁷

This is analogous to systems where append-only logs underpin projections and materialized views, and where consumers can rebuild state by replay. ¹⁸

Mermaid diagrams requested for the paper

Architecture overview:

```

flowchart LR
    subgraph TruthGraph
        TNodes[Artifacts + Events] --> TEdges[Provenance + Temporal + Causal edges]
    end

    subgraph ViewGraph
        VNodes[Nexus nodes + Bundles] --> VEdges[Pipes/Edges with cost, sat, affinity]
    end

    TNodes -->|lossless projection Pi + ledger| VNodes

    subgraph Physics
        Flow[Fluid resource flow] --> Prices[Local prices pi_k]
        Gravity[Gravity maps G_k] --> Routing[Valve scoring + drift]
    end

    VNodes --> Flow
    VEdges --> Flow
    Flow --> Prices
    Gravity --> Routing
    Routing --> Flow

    Presences[Presences (masks + lenses + learning)] -->|emit/absorb| Daimoi[Daimoi (prob packets)]
    Daimoi -->|drift on G_k + cost| ViewGraph
    Prices --> Presences
    Sentinels[Sentinels] -->|trigger| Compactors[Compactors]
    Compactors -->|compress/expand| ViewGraph

```

Absorption + learning loop:

```

sequenceDiagram
    participant d as Daimoi d

```

```

participant p as Presence p
participant g as ViewGraph signals
participant r as Reward/Receipts

d->>p: absorb(d)
p->>p: compute logits = log p_i + beta*s_i + zeta*q_i - lambda*cost_i
p->>p: sample i* ~ softmax(logits / T)
p->>p: store trace (time, logp, xhat, scope)
p->>g: optional action / emission
r-->p: reward r(t)
p->>p: eligibility weight exp(-dt/tau_e)*overlap
p->>p: REINFORCE update w_beta, w_T, u_pk

```

Implementation blueprint and scalability

Storage and schemas

TruthGraph store (immutable): - Nodes: `artifact`, `event`, `bundle_ledger`. - Edges: `provenance`, `temporal_prev`, `causal`, `references`, `projection_links`.

Temporal chains (per stream) support replay and audit; this follows the general log-based “append-only events + projections” paradigm used in large-scale event/log systems. ⁸

ViewGraph store (mutable, derived): - Node table: Nexus/bundle nodes with reservoir states and summary embeddings. - Edge table: pipes with `Fmax`, affinity, health, saturation, cached cost. - Bundle ledger pointers: references into TruthGraph reconstruction artifacts.

Presence state store (per agent): - mask (sparse vector), - influence and pinned nodes, - utilization EMAs and need, - wallet and budgets, - learning params: `w_beta`, `w_T`, adapters `u_pk`, - normalizers (EMA mean/var per feature), - reward baseline (EMA), - trace ring buffer.

Daimoi queue/state: - active particles (sparse, consolidated), - per-edge transit buffers or per-node particle lists, - TTL/hop metadata.

Update triggers and scheduling

- **Fluid tick:** fixed-rate (e.g., 10–60 Hz) or event-driven in active regions.
- **Gravity maps:** recompute on:
 - $\Delta need$ beyond threshold,
 - significant topology change (compress/expand),
 - congestion change beyond threshold,
 - periodic refresh (low frequency) to prevent drift.
- **Compaction/expansion:** Sentinel policies evaluated per region window; Compactor executions rate-limited to avoid thrash.
- **Learning updates:** applied at most every Δt_{learn} per Presence with caps β_{max} , T_{min} , T_{max} .

Complexity and scalability

Let $N = |V^V|$, $E = |E^V|$, $K = |\mathcal{K}|$, $P = |\mathcal{P}|$. Let r be the bounded-radius explored subgraph size in gravity updates.

- Fluid update: $O(EK)$ per tick (can be reduced by simulating only active k per edge).
- Gravity maps: per active Presence $O(r \log r)$ for bounded Dijkstra; multi-source shortest paths are standard. ¹
- Daimoi drift: per hop $O(\deg(n) \cdot K)$, typically small if degree is indirectly bounded via upkeep/health policies.
- Absorption: $O(M)$ where M is component count (kept small by consolidation).

Graph compression reduces N, E in cold regions; summarization and compression literature supports the view that compact representations are essential for large graphs, with WebGraph demonstrating high compression ratios for large sparse graphs under redundancy patterns. ¹⁹

Reproducibility scaffolding

For a publishable artifact, standardize:

- deterministic RNG seeds for Daimoi sampling and routing,
- a “receipt” schema capturing:
 - input Daimoi ids/hashes,
 - path summaries (optional),
 - sampled outcomes,
 - reward assignments and parameter deltas,
 - event-log checkpoints:
- TruthGraph chain heads + digests (Merkle-style optional). ⁹

Experimental evaluation plan

Research questions

- Does gravity + prices reduce congestion and improve time-to-satisfy needs relative to pressure-only routing?
- Do learned lenses produce specialization and better downstream utility vs fixed sampling temperature/selectivity?
- Does lossless compaction preserve correctness while improving runtime scalability without inducing thrash?
- Are congregation patterns and decisions auditable (path- and receipt-explainable) at scale?

Metrics

System efficiency - Utilization per k : mean/p95 of R/Cap and realized throughput. - Congestion: distribution of sat_e , max saturation, congestion duration. - Latency-to-satisfy: time from intent injection to viable completion (e.g., successful “playback complete”).

Stability - Pressure variance over time in hotspots. - Compaction thrash: compress $\leftrightarrow$ expand cycles per region per unit time. - Gravity-map churn: $\|G_k(t) - G_k(t - \Delta)\|$ statistics.

Fairness - Jain’s fairness index across Presences’ achieved allocations or service rates. ²⁰

Semantic/retrieval quality - Retrieval hit@k / recall@k on synthetic labeled topic clusters. - Cluster/community coherence: modularity Q or conductance-based measures (diagnostic, not sole objective). ²¹

Auditability - Explanation size: number of edges/nodes needed to justify a decision (path + local terms). - Replay fidelity: reproduce outputs from receipts and TruthGraph replay.

Synthetic workloads

- **Ingestion streams:** continuous creation of artifact nodes across topic partitions plus event streams.
- **Query/intent bursts:** time-varying hotspots (“play a song” repeated, then shift topic).
- **Cross-basin scarcity:** MP3 nodes exist only in a different basin; measure discovery latency as a function of gate costs and pressure thresholds.
- **Congestion collapse scenarios:** induce path bottlenecks; observe whether prices + routing avoid collapse (compare with classical congestion control behaviors). ²²
- **Compression stress:** large cold background with small hot active region; measure ViewGraph size, expand latency, and correctness.

Ablations

Component removed/ changed	Ablation	Expected observable changes
Gravity wells	$G_k \equiv 0$	weaker congregation, longer time-to-satisfy, more diffusive routing
Learned lens	fix β, T	reduced adaptation to message mixture and context
Prices from pressure	fix π_k constants	more sustained congestion, poorer avoidance response
Sampling collapse	MAP instead of sampling	more deterministic but less exploratory; potential brittleness
Compaction	disable compress/ expand	correctness ok; scalability degraded; less stable runtime

Diagnostics and visualization requests

For the paper and artifact:

- **Heatmaps over the graph:** $P[n, k]$, $\pi_k(n)$, $G_k[n]$, and bundle boundaries (before/after expansion).
- **Edge overlays:** sat_e , $cost(e)$, “river” flows of Daimoi.
- **Time series charts:** utilization, congestion, reward, $\beta(x)$, $T(x)$, compression ratio $|V^V|/|V^T|$.

- **Audit views:** for one decision, show:
 - selected shortest path (edge costs),
 - gravity gradient contribution,
 - sampling logits and realized component,
 - receipt hash.

Design alternatives, priors, and source priorities

Design alternatives table

Choice	Option A	Option B	Notes
Daimoi realization	Sampling	MAP	Sampling supports exploration and learning with stochastic policies; MAP improves determinism but can reduce robustness under ambiguity. ³
Distance metric for gravity	Graph distance	embedding distance	Graph distance is auditable through explicit paths and shortest-path costs. ¹
Categorical implementation	direct categorical	Gumbel-max / relaxations	Gumbel-based methods standardize sampling and (optionally) differentiable relaxations. ²³
ViewGraph compression	lossless ledger	lossy summarization	Lossless enables replay/audit; summarization surveys distinguish task-preserving compression from lossy. ⁶
Congestion response	price + routing	routing-only	Classic congestion control uses signals akin to prices; combining both improves stability. ¹⁴
Coarsening strategy	multilevel	flat clustering	Multilevel partition/coarsening is well established for large irregular graphs. ¹⁵

Initial priors and safety caps

Recommended conservative defaults (tune per workload):

Edge cost - $w_L = 1$, $w_C = 2$, $w_S = 1$

Gravity - sources per Presence $S = 4$ - radius $R_{cost} = 6$ (or hop radius 3) - $need_{min} = 0.05$ - cap potentials: $\Phi_p(n) \leq \Phi_{max}$ and/or cap $Mass_p$

Pricing - scarcity sensitivity κ_k : compute higher than RAM, I/O moderate (start around 0.8–1.6) - congestion premium ρ_k : NET/DISK higher than compute (start around 0.2–0.6) - adapter bound $u_{max} = 0.5$ ($\approx \times 1.65$)

Lens learning - $\beta_{max} = 6$ - $T_{min} = 0.2$, $T_{max} = 2.0$ - learning rate η small (e.g., 1e-3–1e-2 for linear heads) - trace TTL 30–120s; ring buffer capped - baseline EMA and variance normalization

These caps are aligned with known practice in stabilizing stochastic-policy learning and avoiding extreme temperatures/selectivities. ²⁴

Source priorities for the related work section

A publishable paper should ground claims in primary references:

Shortest paths and auditable distance - Foundational shortest-path formulation. ¹

Congestion control and network pricing - Congestion avoidance/control (foundational). ²⁵

- Shadow prices and proportional fairness / stability (network utility framing). ²⁶

Policy gradient learning and online decision-making - REINFORCE (policy gradient for stochastic units). ²⁷

- Eligibility traces and stabilization (textbook reference). ²⁸

- Bandit algorithms for online adaptation and nonstationarity. ¹³

Categorical sampling and relaxations - Gumbel-max survey and extensions. ²⁹

- Gumbel-Softmax / Concrete relaxations. ³⁰

Graph summarization, compression, and coarsening - Graph summarization survey taxonomy. ³¹

- Large sparse graph compression techniques (WebGraph). ³²

- Multilevel coarsening/partitioning methodology (METIS lineage). ¹⁵

Append-only event logs and replayable projections - Distributed log for event ingestion and replay-style consumption. ³³

- Append-only event log processing rationale and projections (overview). ⁷

- Merkle-style digests for verifiable chain/window integrity. ⁹

Experiment timeline

Phase	Deliverables	Typical duration
Instrumentation	receipts, trace logs, gravity/price snapshots, replay harness	1–2 weeks
Baselines	core workloads running, stable dashboards	1 week
Ablations	no-gravity, fixed-lens, fixed-price, MAP collapse	1–2 weeks
Scaling	N , E sweeps, radius sensitivity, compression overhead	1–2 weeks
Paper figures	architecture diagram, heatmaps, audit-case studies	1 week
Write-up	formal model + algorithms + results + appendix	2–3 weeks

¹ ⁵ A Note on Two Problems in Connexion with Graphs

https://ir.cwi.nl/pub/9256/9256D.pdf?utm_source=chatgpt.com

2 26 shadow prices, proportional fairness and stability

https://www.statslab.cam.ac.uk/~frank/rate.pdf?utm_source=chatgpt.com

3 12 27 Simple Statistical Gradient-Following Algorithms for

https://www-anw.cs.umass.edu/~barto/courses/cs687/williams92simple.pdf?utm_source=chatgpt.com

4 8 18 33 Kafka: a Distributed Messaging System for Log Processing

https://notes.stephenholiday.com/Kafka.pdf?utm_source=chatgpt.com

6 31 Graph Summarization Methods and Applications: A Survey

https://gemslab.github.io/papers/liu-2018-graph-summarization.pdf?utm_source=chatgpt.com

7 Online Event Processing

https://martin.kleppmann.com/papers/olep-cacm.pdf?utm_source=chatgpt.com

9 17 A Digital Signature Based on a Conventional Encryption ...

https://link.springer.com/chapter/10.1007/3-540-48184-2_32?utm_source=chatgpt.com

10 23 29 A Review of the Gumbel-max Trick and its Extensions for Discrete Stochasticity in Machine Learning

https://arxiv.org/abs/2110.01515?utm_source=chatgpt.com

11 24 28 Reinforcement Learning: An Introduction

https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf?utm_source=chatgpt.com

13 Bandit Algorithms - tor-lattimore.com

https://tor-lattimore.com/downloads/book/book.pdf?utm_source=chatgpt.com

14 22 25 Congestion Avoidance and Control

https://cs162.org/static/readings/jacobson-congestion.pdf?utm_source=chatgpt.com

15 A fast and high quality multilevel scheme for partitioning ...

https://www.cs.utexas.edu/~pingali/CS395T/2009fa/papers/metis.pdf?utm_source=chatgpt.com

16 Event Sourcing pattern - Azure Architecture Center

https://learn.microsoft.com/en-us/azure/architecture/patterns/event-sourcing?utm_source=chatgpt.com

19 32 The WebGraph Framework I: Compression Techniques*

https://vigna.di.unimi.it/ftp/papers/WebGraphI.pdf?utm_source=chatgpt.com

20 A Quantitative Measure Of Fairness And Discrimination For Resource Allocation In Shared Computer Systems

https://arxiv.org/abs/cs/9809099?utm_source=chatgpt.com

21 Finding and evaluating community structure in networks

https://www.ccs.neu.edu/home/vip/teach/DMcourse/6_graph_analysis/notes_slides/PhysRevE.69.026113.pdf?utm_source=chatgpt.com

30 CATEGORICAL REPARAMETERIZATION

https://openreview.net/pdf?id=rkE3y85ee&utm_source=chatgpt.com